

Parcial 2 – Ejercicio de Prueba

Ejercicio 1:

En los sistemas embebidos modernos, como los utilizados en estaciones meteorológicas o plantas industriales, es común que múltiples sensores envíen lecturas periódicas. Para mantener eficiencia, un microcontrolador almacena estas lecturas (temperaturas) en un **Árbol Binario de Búsqueda (ABB)**.

Vas a implementar una clase `RegistroSensoresABB` que permita:

1. **Insertar temperaturas** registradas en el ABB.
 2. Usar recursividad para **contar cuántas temperaturas son iguales o mayores a cierto umbral** (por ejemplo, para detectar condiciones críticas).
 3. Imprimir directamente ese conteo en un formato predefinido.
-

Detalles importantes:

- Implementá un método `insertar(int temperatura)` para almacenar una lectura.
- Implementá el método recursivo `contarCriticos(int umbral)` para contar valores mayores o iguales al umbral.
- Implementá el método `imprimirCriticos(int umbral)` que imprima exactamente:

```
Sensores criticos (temp >= <umbral>): <cantidad>
```

```
#include <iostream>
```

```
class RegistroSensoresABB {  
public:
```

```
    void insertar(int temperatura) {  
        // Implementar  
    }
```

```
    int contarCriticos(int umbral) {  
        // Implementar recursivamente  
        return -1;  
    }
```

```
    void imprimirCriticos(int umbral) {  
        // Usar contarCriticos e imprimir en el formato requerido  
    }
```

```
private:
```

```
    // Agregar estructuras auxiliares (Nodo, punteros, etc.)  
};
```

SOLUCION

Ejercicio 2

Durante la misión CONAE-Marte, la nave y sus módulos auxiliares (sensores térmicos, cámaras, actuadores, procesadores) se comunican entre sí a través de buses de datos (como CAN o I2C). Para mantener un funcionamiento seguro y eficiente, es necesario analizar cómo están **conectados físicamente** entre ellos.

Cada módulo es un **nodo**, y cada conexión directa es una **arista**. Estas conexiones forman una **red no dirigida** que debe ser modelada para realizar diagnósticos en vuelo.

Implementará una clase `RedDeSensores` que permita:

- `agregarConexion(mod1, mod2)`: agregar una conexión directa bidireccional entre módulos.
- `estanConectados(mod1, mod2)`: devolver `true` si `mod1` y `mod2` tienen una conexión directa.
- `alcanzablesDesde(modInicio)`: devolver cuántos módulos pueden alcanzarse desde `modInicio`, utilizando búsqueda DFS recursiva.

(sistema de penalización: 0 %)

```
#include <iostream>
#include <map>
#include <vector>
#include <string>
#include <set>
using namespace std;

class RedDeSensores {
private:
    map<string, vector<string>>> grafo;

public:
    // Agrega una conexión bidireccional entre dos módulos
    void agregarConexion(const string& a, const string& b) {
        // TBD
    }

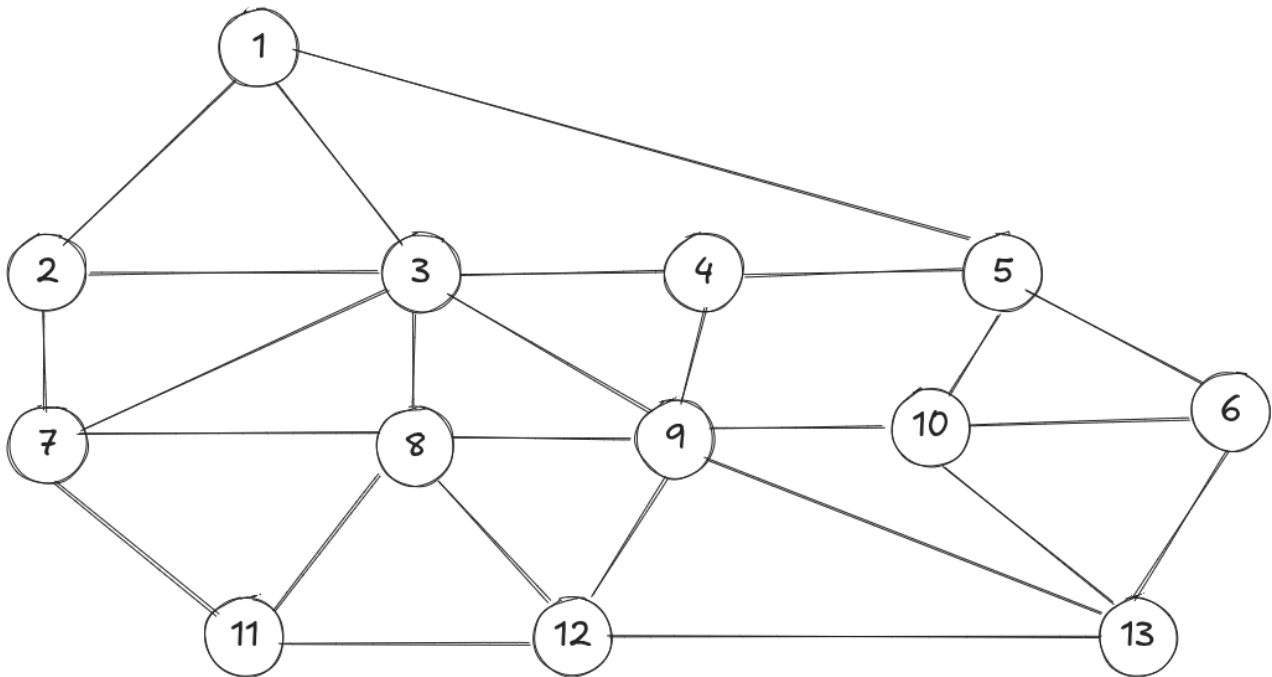
    // Devuelve true si existe conexión directa entre a y b
    bool estanConectados(const string& a, const string& b) {
        // TBD
    }

    // Devuelve cuántos módulos son alcanzables desde el módulo de inicio (incluido), usando DFS
    int alcanzablesDesde(const string& inicio) {
        // TBD
    }

private:
    void dfs(const string& actual, set<string>& visitados) {
        // TBD
    }
}
```

};

Ejercicio 3 – Teorico



Si se hace un recorrido en profundidad (DFS), el recorrido de los nodos, a partir del nodo origen **1**, es...

Pregunta 3 Respuesta

a.

1 10 5 9 12 11 7 8 3 13 6 4 2

b.

1 5 10 9 12 11 7 8 3 13 6 4 2

c.

1 2 10 9 12 11 7 8 3 13 6 4 5

d.

1 5 10 9 12 11 7 8 3 13 4 6 2 (MARQUE ESTA OPCION, INCORRECTO)

Ejercicio 4 – Teorico

En una lista simplemente enlazada (solo un puntero a la cabeza de lista) usada para llevar el registro de turnos médicos en un sanatorio (FIFO), eliminar el último turno agendado tiene complejidad $O(1)$, es decir, es constante.

Pregunta 4 Respuesta FALSO

Verdadero

Falso

Ejercicio 5 – Teorico

Una **cola de prioridad** sería adecuada para organizar las órdenes de entrega en una empresa de logística según urgencia.

Pregunta 5 Respuesta VERDADERO

Verdadero

Falso